

Ex 26.

```
from google import genai

client =
genai.Client(api_key="AQ.Ab8RN6JTxJSaFWKkSlpFiHg3SWr55JF8tnO_eISeC9xF0YypfA")

prompt = input("Enter your prompt: ")

response = client.models.generate_content(

    model="gemini-3.5-flash",

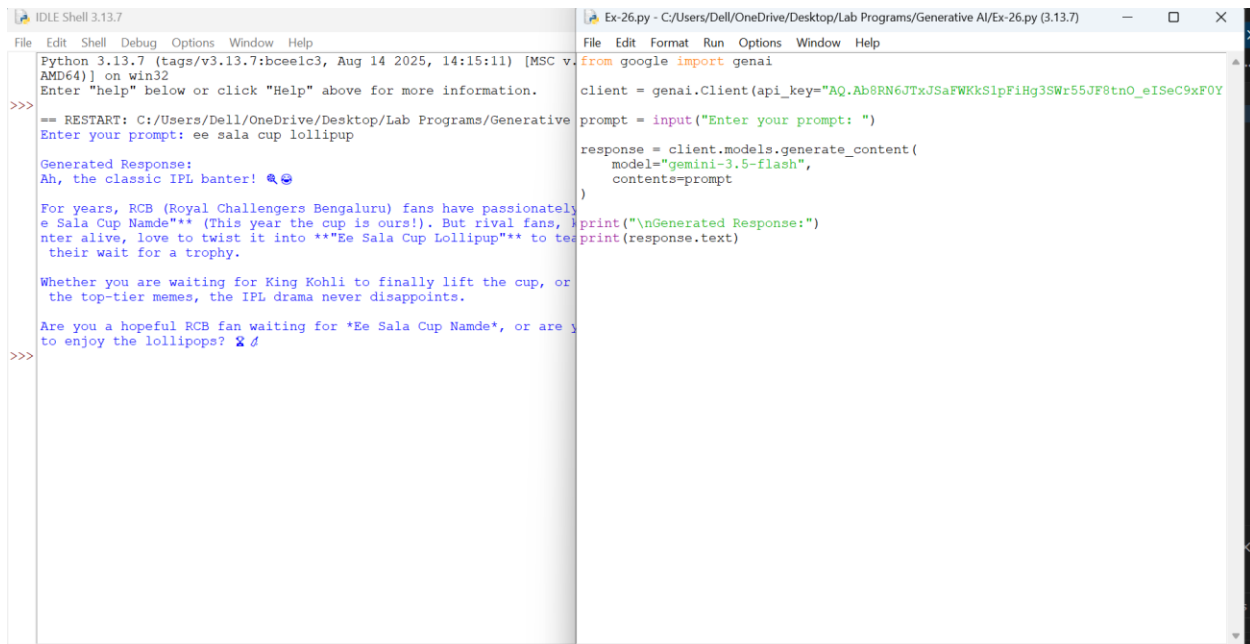
    contents=prompt

)

print("\nGenerated Response:")

print(response.text)
```

Output:



```
Python 3.13.7 (tags/v3.13.7:bce1c3, Aug 14 2025, 14:15:11) [MSC v.1916 AMD64] on win32
File Edit Shell Debug Options Window Help
>>> Enter "help" below or click "Help" above for more information.
== RESTART: C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/Ex-26.py
Enter your prompt: ee sala cup lollipop

Generated Response:
Ah, the classic IPL banter! 🍭

For years, RCB (Royal Challengers Bengaluru) fans have passionately
e Sala Cup Namde" (This year the cup is ours!). But rival fans,
nter alive, love to twist it into "Ee Sala Cup Lollipop" to tease
their wait for a trophy.

Whether you are waiting for King Kohli to finally lift the cup, or
the top-tier memes, the IPL drama never disappoints.

Are you a hopeful RCB fan waiting for "Ee Sala Cup Namde", or are y
to enjoy the lollipops? 🍭

>>>
```

```
Ex-26.py - C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/Ex-26.py (3.13.7)
File Edit Format Run Options Window Help
from google import genai
client = genai.Client(api_key="AQ.Ab8RN6JTxJSaFWKkSlpFiHg3SWr55JF8tnO_eISeC9xF0YypfA")
prompt = input("Enter your prompt: ")
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt
)
print("\nGenerated Response:")
print(response.text)
```

Ex 27:

```
from google import genai

# Gemini API key

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

# Get prompt from user

prompt = input("Enter your prompt: ")

# Generate response

response = client.models.generate_content(

    model="gemini-3.5-flash",

    contents=prompt

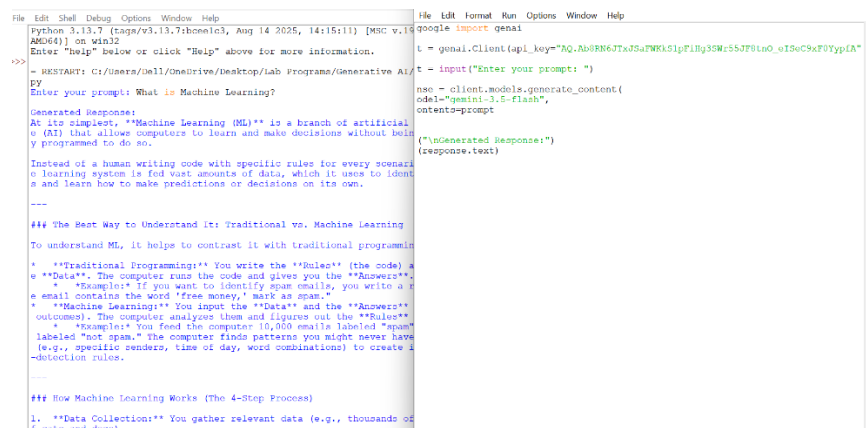
)

# Display output

print("\nGenerated Output:")

print(response.text)
```

Output:



```
File Edit Shell Debug Options Window Help File Edit Format Run Options Window Help
Python 3.13.7 (tags/v3.13.7:hceec1c3, Aug 14 2025, 14:15:11) [MSC v.19
AM064] on win32
>>>
-- RESTART: C:/Users/um1/OneDrive/Desktop/Lab Programs/Generative AI/
PY
Enter your prompt: What is Machine Learning?
Generated Response:
At its simplest, "Machine Learning (ML)" is a branch of artificial
intelligence (AI) that allows computers to learn and make decisions without being
explicitly programmed to do so.

Instead of a human writing code with specific rules for every scenario, a
machine learning system is fed vast amounts of data, which it uses to identify
patterns and learn how to make predictions or decisions on its own.

---

## The Best Way to Understand It: Traditional vs. Machine Learning

To understand ML, it helps to contrast it with traditional programming.

* "Traditional Programming:" You write the "Rules" (the code) and
provide "Data". The computer runs the code and gives you the "Answers".
* Example: If you want to identify spam emails, you write a rule that
if an email contains the word "free money," mark it as spam.
* "Machine Learning:" You input the "Data" and the "Answers" (the
outcomes). The computer analyzes them and figures out the "Rules" for
you.
* Example: You feed the computer 10,000 emails labeled "spam" or
"not spam." The computer finds patterns you might never have
(e.g., specific senders, time of day, word combinations) to create its
own detection rules.

---

## How Machine Learning Works (The 4-Step Process)

1. "Data Collection:" You gather relevant data (e.g., thousands of
emails and their labels).
```

Ex 28:

```
from huggingface_hub import InferenceClient

# Your Hugging Face token

HF_TOKEN = "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"

# Create client

client = InferenceClient(

    provider="auto",

    api_key=HF_TOKEN

)

# Get user prompt

prompt = input("Enter your prompt: ")

# Generate text

response = client.chat.completions.create(

    model="deepseek-ai/DeepSeek-V3-0324",

    messages=[

        {

            "role": "user",

            "content": prompt

        }

    ],

    max_tokens=200

)

print("\n===== GENERATED TEXT =====")

print(response.choices[0].message.content)
```

Output:

```
File Edit Shell Debug Options Window Help
File "C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/GE
, line 9, in <module>
    result = client.text_generation(
File "C:/Users/Dell/AppData/Local/Programs/Python/Python313\Lib\
huggingface_hub\inference\_client.py", line 2383, in text_generation
    provider_helper = get_provider_helper(self.provider, task="text"
model=model_id)
File "C:/Users/Dell/AppData/Local/Programs/Python/Python313\Lib\ai
huggingface_hub\inference\_providers\__init__.py", line 240, in get_
per
    provider = next(iter(provider_mapping)).provider
StopIteration
>>>
= RESTART: C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI
PY
Enter your prompt: Artificial Intelligence is

===== GENERATED TEXT =====
**Artificial Intelligence (AI)** refers to the simulation of human i
in machines, enabling them to perform tasks that typically require h
on, such as:

- **Learning** (acquiring information and rules)
- **Reasoning** (applying rules to reach conclusions)
- **Problem-solving**
- **Perception** (e.g., computer vision, speech recognition)
- **Decision-making**

### **Types of AI**
1. **Narrow AI (Weak AI)** - Specialized in one task (e.g., chatbots
ognition).
2. **General AI (Strong AI)** - Hypothetical AI with human-like reas
et achieved).
3. **Superintelligent AI** - Surpasses human intelligence (theoretic

### **Key Techniques**
- **Machine Learning (ML)** - Algorithms learn from data (e.g., deep
eural networks).
- **Natural Language Processing (NLP)** - En

>>>
File Edit Format Run Options Window Help
from huggingface_hub import InferenceClient

# Your Hugging Face token
HF_TOKEN = "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"

# Create client
client = InferenceClient(
    provider="auto",
    api_key=HF_TOKEN
)

# Get user prompt
prompt = input("Enter your prompt: ")

# Generate text
response = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V3-0324",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ],
    max_tokens=200
)

print("\n===== GENERATED TEXT =====")
print(response.choices[0].message.content)
```

Ex 29:

from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

problem = input("Enter the computational problem: ")

prompt = f"""

You are a Python programming expert.

TASK:

Generate a Python program for the following problem:

{problem}

REQUIREMENTS:

1. Use Python.
2. Keep the code simple.
3. Add comments explaining important steps.
4. Use meaningful variable names.

5. The program should accept user input if required.
6. Provide the expected output format.

OUTPUT FORMAT:

1. Problem Explanation
2. Python Code
3. Sample Input
4. Sample Output

```
"""

response = client.models.generate_content(

    model="gemini-3.5-flash",

    contents=prompt

)

print("\n===== GENERATED PYTHON PROGRAM =====")

print(response.text)
```

Output:

The screenshot shows a Python IDE with two panes. The left pane displays the output of a program that generates a Python code snippet for a student grade calculator. The right pane shows the Python code being generated by the AI.

Left Pane (Generated Code):

```
Python 3.13.7 (tags/v3.13.7:bceec1c3, Aug 14 2025, 14:15:11) [MSC v.194
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/
Enter the computational problem: Student Problem

===== GENERATED PYTHON PROGRAM =====
### 1. Problem Explanation

**The Student Grade Calculator Problem:**
The goal of this program is to calculate a student's final results bas
r marks in three key subjects: **Math**, **Science**, and **English**.

The program will:
1. Accept the student's name and marks for the three subjects from the
2. Calculate the **total marks** and the **average percentage score**.
3. Assign a letter **grade** based on the following criteria:
   - Average Score >= 90%: **Grade A**
   - Average Score >= 80% and < 90%: **Grade B**
   - Average Score >= 70% and < 80%: **Grade C**
   - Average Score >= 60% and < 70%: **Grade D**
   - Average Score < 60%: **Grade F**
4. Display a clean and formatted "Student Report Card".

---

### 2. Python Code

```python
Program: Student Grade Calculator

Step 1: Accept student details and subject marks from the user
student_name = input("Enter the student's name: ")

Convert inputs to float to allow decimal marks
math_score = float(input("Enter marks obtained in Math (out of 100): ")
science_score = float(input("Enter marks obtained in Science (out of 1
english_score = float(input("Enter marks obtained in English (out of 1

===== GENERATED PYTHON PROGRAM =====
print(response.text)
```

**Right Pane (Python Code):**

```
tx-z9.py - C:/Users/ueh/OneDrive/Desktop/Lab Programs/Generative AI/tx-z9.py (3.13.7)
File Edit Format Run Options Window Help

from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtdlHrStvFjA"

client = genai.Client(api_key=API_KEY)

problem = input("Enter the computational problem: ")

prompt = f"""
You are a Python programming expert.

TASK:
Generate a Python program for the following problem:

{problem}

REQUIREMENTS:
1. Use Python.
2. Keep the code simple.
3. Add comments explaining important steps.
4. Use meaningful variable names.
5. The program should accept user input if required.
6. Provide the expected output format.

OUTPUT FORMAT:
1. Problem Explanation
2. Python Code
3. Sample Input
4. Sample Output
"""

response = client.models.generate_content(
 model="gemini-3.5-flash",
 contents=prompt
)

print("\n===== GENERATED PYTHON PROGRAM =====")
print(response.text)
```

**Ex 30:**

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

schema = """

Database: CollegeDB

Table: Students

Columns:

student_id INT

name VARCHAR(100)

age INT

department VARCHAR(50)

marks INT

"""

requirement = input("Enter your SQL requirement: ")

prompt = f"""

You are an SQL expert.

DATABASE SCHEMA:

{schema}

REQUIREMENT:

{requirement}

INSTRUCTIONS:

1. Generate the appropriate SQL query.

2. Use only the tables and columns provided.

3. Do not invent columns.

4. Keep the query simple and correct.

5. Explain the query briefly.
```

OUTPUT FORMAT:

SQL QUERY:

<SQL query>

EXPLANATION:

<short explanation>

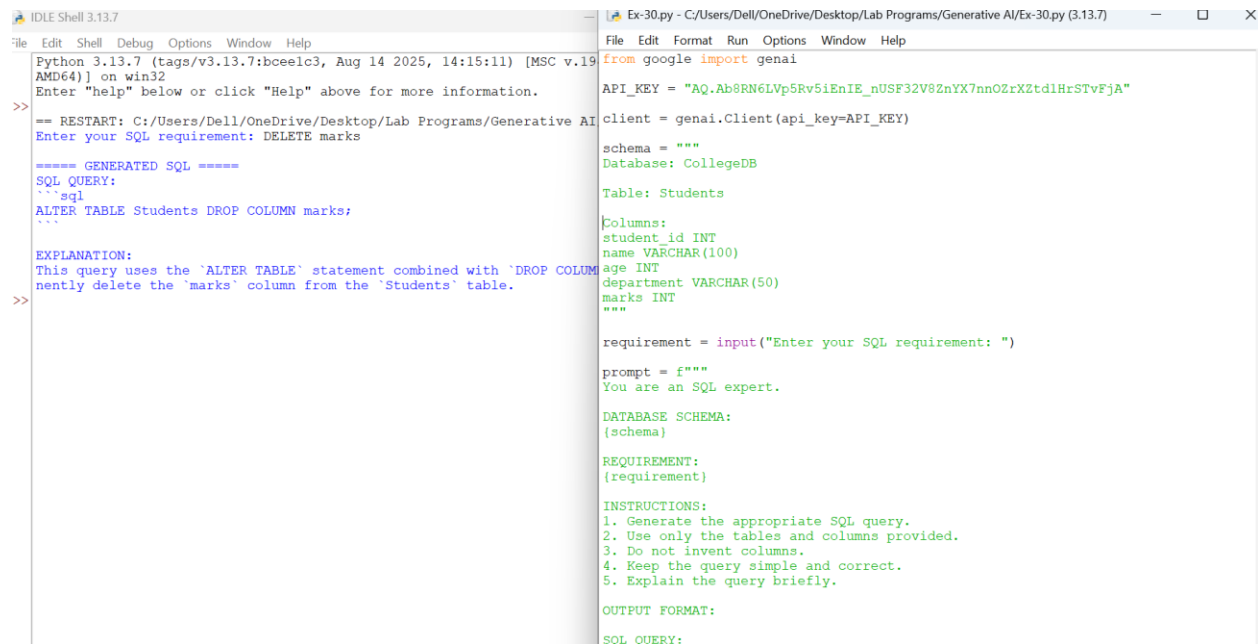
"""

```
response = client.models.generate_content(
 model="gemini-3.5-flash",
 contents=prompt
)

print("\n===== GENERATED SQL =====")

print(response.text)
```

## Output:



```
IDLE Shell 3.13.7
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1916 AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/Ex-30.py (3.13.7) ==
Enter your SQL requirement: DELETE marks

===== GENERATED SQL =====
SQL QUERY:
```sql
ALTER TABLE Students DROP COLUMN marks;
```

EXPLANATION:
This query uses the 'ALTER TABLE' statement combined with 'DROP COLUMN' to delete the 'marks' column from the 'Students' table.
>>>
```

```
Ex-30.py - C:/Users/Dell/OneDrive/Desktop/Lab Programs/Generative AI/Ex-30.py (3.13.7)
File Edit Format Run Options Window Help
from google import genai
API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnO2rXZtdlHrStvFjA"
client = genai.Client(api_key=API_KEY)

schema = """
Database: CollegeDB

Table: Students

Columns:
student_id INT
name VARCHAR(100)
age INT
department VARCHAR(50)
marks INT
"""

requirement = input("Enter your SQL requirement: ")

prompt = f"""
You are an SQL expert.

DATABASE SCHEMA:
{schema}

REQUIREMENT:
{requirement}

INSTRUCTIONS:
1. Generate the appropriate SQL query.
2. Use only the tables and columns provided.
3. Do not invent columns.
4. Keep the query simple and correct.
5. Explain the query briefly.

OUTPUT FORMAT:

SQL QUERY:
```

**Ex 31:**

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):

 response = client.models.generate_content(

 model="gemini-3.5-flash",

 contents=prompt

)

 return response.text

task = """

Explain the benefits of Artificial Intelligence in education

in a short paragraph.

"""

ZERO-SHOT

zero_shot = f"""

{task}

Use simple and clear language.

"""

ONE-SHOT

one_shot = f"""

{task}

Example:

Topic: Benefits of Computers in Education

Example Response:

Computers help students access information quickly, learn through

interactive materials, complete assignments, and improve digital
```



skills.

Now answer the given task in a similar style.

"""

# FEW-SHOT

few\_shot = f"""

{task}

Example 1:

Topic: Benefits of Computers in Education

Response:

Computers help students access information quickly, use educational software, complete assignments, and develop digital skills.

Example 2:

Topic: Benefits of Online Learning

Response:

Online learning provides flexible access to educational materials, allows students to learn at their own pace, and supports learning from different locations.

Now answer the task using the same style and structure.

"""

```
print("\n===== ZERO-SHOT =====")
```

```
result_zero = generate(zero_shot)
```

```
print(result_zero)
```

```
print("\n===== ONE-SHOT =====")
```

```
result_one = generate(one_shot)
```

```
print(result_one)
```

```
print("\n===== FEW-SHOT =====")
```

```
result_few = generate(few_shot)
```

```
print(result_few)

Comparison

print("\n===== COMPARISON =====")

print("""
```

| Criteria    | Zero-shot | One-shot  | Few-shot  |
|-------------|-----------|-----------|-----------|
| -----       |           |           |           |
| Quality     | Good      | Very Good | Excellent |
| Relevance   | Good      | Very Good | Excellent |
| Accuracy    | Good      | Very Good | Excellent |
| Consistency | Moderate  | Good      | Very Good |
| -----       |           |           |           |

Conclusion:

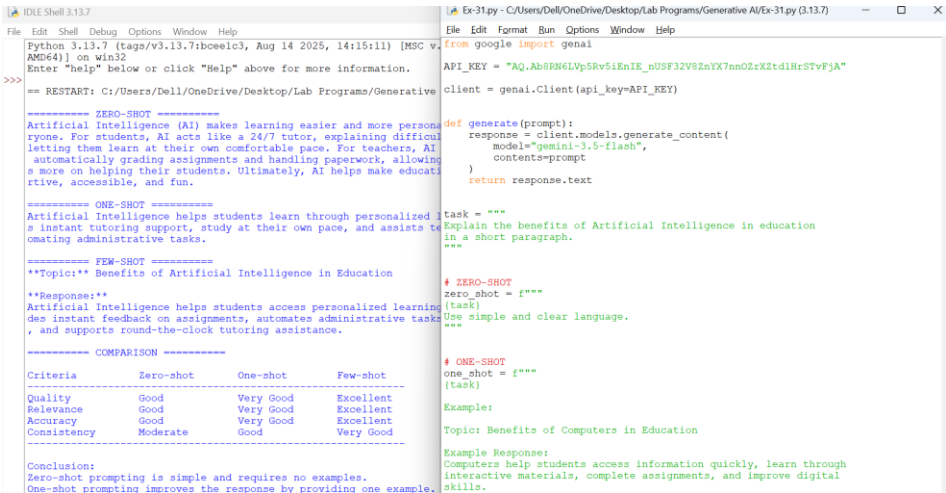
Zero-shot prompting is simple and requires no examples.

One-shot prompting improves the response by providing one example.

Few-shot prompting generally provides the most consistent response because multiple examples guide the model.

""")

Output:



**Ex 32:**

```
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtd1HrSTvFjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):

 response = client.models.generate_content(

 model="gemini-3.5-flash",

 contents=prompt

)

 return response.text

task = """

Explain the advantages of Artificial Intelligence in healthcare.

"""

Prompt 1

prompt1 = f"""

{task}

Write a short answer.

"""

Prompt 2

prompt2 = f"""

{task}

Explain the answer using:

1. Introduction

2. Applications

3. Benefits

4. Conclusion
```

Use simple language.

"""

# Prompt 3

prompt3 = f"""

You are an AI and healthcare expert.

TASK:

{task}

REQUIREMENTS:

- Use simple language.
- Explain at least three advantages.
- Give practical examples.
- Be accurate.
- Avoid unnecessary information.
- Use headings.
- End with a conclusion.
- Keep the answer between 150 and 200 words.

OUTPUT FORMAT:

Introduction

Advantages

Examples

Conclusion

"""

# Generate responses

result1 = generate(prompt1)

result2 = generate(prompt2)

result3 = generate(prompt3)

```
print("\n===== PROMPT 1 =====")
print(result1)
print("\n===== PROMPT 2 =====")
print(result2)
print("\n===== PROMPT 3 =====")
print(result3)
```

# Evaluation prompt

```
evaluation_prompt = f"""
```

Evaluate the following three responses.

TASK:

```
{task}
```

RESPONSE 1:

```
{result1}
```

RESPONSE 2:

```
{result2}
```

RESPONSE 3:

```
{result3}
```

Evaluate each response using:

1. Relevance
2. Accuracy
3. Completeness
4. Clarity
5. Format adherence

Give each criterion a score from 1 to 5.

Then identify the best-performing response.

Finally, create a refined prompt that would produce an even better answer.

Use this format:

RESPONSE 1 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

RESPONSE 2 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

RESPONSE 3 SCORE:

Relevance:

Accuracy:

Completeness:

Clarity:

Format:

BEST RESPONSE:

...

REFINED PROMPT:

...

"""

evaluation = generate(evaluation\_prompt)

```
print("\n===== EVALUATION =====")

print(evaluation)
```

Output:

```

le Edit Shell Debug Options Window Help
#####
4. Conclusion

Artificial Intelligence is one of the most exciting tools in modern medicine. While it will never replace the care, empathy, and touch of a real doctor, it is an incredibly powerful helper. By making medical care faster, more accurate, and more personalized, AI is helping people all over the world live healthier lives.

===== PROMPT 3 =====
Introduction
Artificial Intelligence (AI) is rapidly transforming modern healthcare, making medical processes faster, more accurate, and highly efficient.

Advantages
Here are three key advantages of AI in healthcare:
1. **Faster Diagnosis:** AI analyzes complex medical data rapidly, helping doctors detect deadly diseases much earlier.
2. **Personalized Treatment:** It designs specific, effective therapies based on a patient's unique genetic profile and medical history.
3. **Increased Efficiency:** AI automates repetitive administrative tasks, reducing paperwork and allowing doctors to spend more quality time with patients.

Examples
***Cancer Screening:** AI algorithms analyze mammograms to spot potential cancer tumors that human eyes might easily miss.
***Drug Discovery:** AI quickly identifies promising chemical compounds, significantly reducing the time needed to develop new, life-saving medications.
***Virtual Nurses:** AI-powered chatbots answer patient questions, provide health advice, and manage medication schedules 24/7.

Conclusion
In conclusion, AI in healthcare saves valuable lives, reduces overhead costs, and significantly improves patient care worldwide.

===== EVALUATION =====

File Edit Format Run Options Window Help
from google import genai

API_KEY = "AQ.Ab8RN6LVp5Rv5iEnIE_nUSF32V8ZnYX7nnOZrXZtdlHrStvfjA"

client = genai.Client(api_key=API_KEY)

def generate(prompt):
 response = client.models.generate_content(
 model="gemini-3.5-flash",
 contents=prompt
)
 return response.text

task = """
Explain the advantages of Artificial Intelligence in healthcare.
"""

Prompt 1
prompt1 = f"""
{task}
Write a short answer.
"""

Prompt 2
prompt2 = f"""
{task}
Explain the answer using:
1. Introduction
2. Applications
3. Benefits
4. Conclusion
"""

```